

Reporte al equipo IA · Podcast del Aula Virtual

Fecha: 2026-09-21 (actualizado 2026-09-23) • Curso probado: Policía Local de Galicia · temas 3 y 10

Endpoints afectados: POST `https://ia.opox.ai/v1/classroom/podcast` (generación) y GET

`https://ia.opox.ai/v1/classroom/podcast/{file}` (audio servido)

Origen del reporte: pruebas en dispositivo del módulo Podcast, reproducible de forma consistente.

Resumen

Estado actualizado tras verificación en producción (2026-09-23):

- **Bug 1 — Seek roto: PARCIALMENTE resuelto.** El endpoint ya soporta HTTP Range (206 correcto, confirmado con `curl`). Pero encontramos una **segunda causa, independiente**: el mp3 que genera el Motor no trae cabecera `Xing/VBRI`, así que los reproductores nativos (ExoPlayer en Android, y probablemente AVPlayer en iOS) no pueden calcular a qué byte saltar en un archivo de bitrate variable, aunque el servidor sí soporte Range. El seek sigue sin funcionar en el dispositivo por esta segunda causa.
- **Bug 2 — Duración real vs. pedida: prácticamente resuelto.** Verificamos una generación reciente con `duracion: 'corta' → duration_seconds: 303.84` contra un `target_seconds: 300` (0.9% de desvío, antes era 1.6-1.7× más largo). Gran mejora.

De nuestro lado ya desplegamos el proxy (GET `/tutor/podcast/audio/:filename`) reenviando Range y el método HTTP real (HEAD incluido — antes siempre hacíamos GET, lo que le hacía perder a Node las cabeceras `Content-Length/Accept-Ranges` en las respuestas a HEAD). Confirmado con `curl` en producción: HEAD y GET con Range responden correctamente. No requiere más cambios de nuestro lado para el soporte de Range — pero el seek sigue bloqueado por la falta de cabecera `Xing/VBRI` en el archivo.

Bug 1 — El usuario no puede adelantar ni retroceder el audio

Cuando el usuario abre un podcast generado y pulsa “15s atrás / 15s adelante”, no pasa nada. Tampoco puede saltar tocando la barra de progreso — la reproducción solo avanza en línea, de principio a fin.

Causa original

El endpoint GET `/v1/classroom/podcast/{file}` respondía 405 Method Not Allowed a peticiones HEAD y no exponía las cabeceras `Accept-Ranges: bytes` ni `Content-Length`. Sin eso, los reproductores nativos de Android (ExoPlayer) e iOS (AVPlayer) no podían hacer seek sobre un mp3 remoto.

De nuestro lado (ya desplegado): nuestro proxy GET `/tutor/podcast/audio/:filename` (necesario porque `expo-audio` no puede mandar el header `X-API-Key` que exige el Motor) tampoco reenviaba el header Range del cliente ni el 206/Content-Range de vuelta. Ya lo corregimos: el proxy ahora reenvía Range al Motor y hace pass-through de Accept-Ranges / Content-Length / Content-Range cuando el Motor los manda. Verificado en producción con `curl`, ambos casos responden bien.

Bug 1b — Nueva causa encontrada: falta cabecera Xing/VBRI en el mp3 (2026-09-23)

Con el soporte de Range ya funcionando en el servidor, el seek **seguía sin funcionar** en dispositivo (Android). Descargamos los primeros 8 KB de un mp3 generado y confirmamos que **no contiene ninguna de las cabeceras Xing, Info ni VBRI** — el marcador que los decoders usan para saber, en un archivo de bitrate variable (VBR), a qué byte exacto corresponde cada segundo de audio.

Sin esa cabecera, ExoPlayer (Android) y probablemente AVPlayer (iOS) no pueden calcular una posición de seek fiable — es una limitación conocida de esos frameworks con mp3 VBR sin seek table, independiente de si el servidor soporta HTTP Range o no. Por eso el seek sigue bloqueado aunque ya arreglamos la parte del servidor.

Lo que necesitamos

Que el mp3 se codifique con la cabecera Xing/VBRI incluida (la mayoría de encoders, incluyendo LAME, la agregan automáticamente si no se desactiva explícitamente; en `ffmpeg` con `libmp3lame` normalmente también se escribe salvo que el pipeline la esté recortando después). Alternativa más simple si el pipeline ya genera CBR (bitrate constante): declararlo explícitamente en vez de VBR — con CBR los decoders pueden estimar la posición de seek por bitrate fijo sin necesitar la cabecera.

Cómo verificar

Los primeros ~1-2 KB del archivo deben contener la cadena ASCII `Xing`, `Info` o `VBRI` poco después del primer frame header MP3 (`0xFF 0xFx`).

Bug 2 — La duración real del audio no coincide con la pedida

En la app el usuario elige entre “Corta (~5 min)” y “Media (~10 min)”. Enviamos ese valor tal cual en el campo `duracion` del `PodcastIn` (`'corta'` o `'media'`).

Lo que medíamos en dispositivo originalmente (velocidad 1.0x, curso Policía Local de Galicia, temas 3 y 10):

duracion enviada	Target	Duración real del mp3	Desvío
corta	~5 min	8:35	1.72x más largo
media	~10 min	16:07	1.61x más largo

Actualización 2026-09-23: generamos un podcast nuevo (`duracion: 'corta'`, mismo curso) y medimos `duration_seconds: 303.84` contra `target_seconds: 300` — **0.9% de desvío**, dentro de lo razonable. Parece resuelto o muy mejorado. Dejamos la tabla y las sospechas originales como referencia por si el desvío grande reaparece en otras configuraciones (voz/velocidad distintas, temas más largos, etc.).

De nuestro lado ya usamos la duración real que ustedes devuelven ([duration_seconds](#)) en vez de un estimado fijo — antes lo ignorábamos y siempre most&raacute;bamos 300s/600s según el tier pedido.

Cómo reproducir

- Generar un podcast desde la app (Aula Virtual → Podcast) con [duracion](#): 'corta' o 'media', cualquier tema del curso Policía Local de Galicia.
- Medir la duración real del mp3 devuelto vs. el target.
- Intentar hacer seek (arrastrar la barra o pulsar $\pm 15s$) sobre un episodio ya generado — seguirá sin funcionar en Android/ExoPlayer hasta que el mp3 traiga cabecera Xing/VBRI (o sea CBR).
- Para confirmar la falta de cabecera: descargar los primeros 2-8 KB del mp3 y buscar las cadenas [Xing](#), [Info](#) o [VBRI](#).

Contacto

Cualquier cosa que necesiten de nuestro lado (payloads exactos, logs, más mediciones) la pasamos sin problema.